

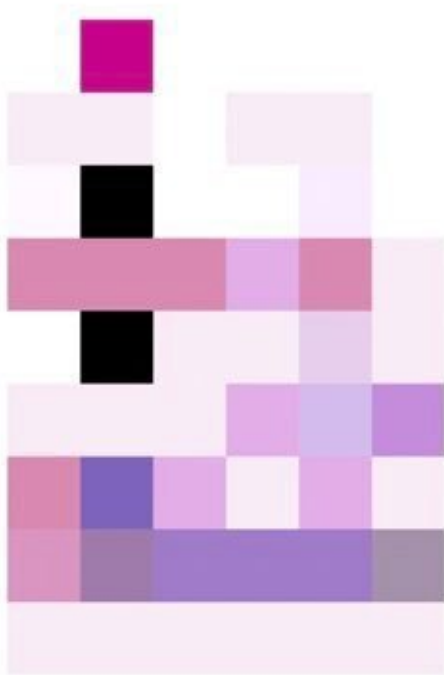
3>SHAKE

型は壁、Rustでもバグを
直すな、表現できなくせよ

`is_paid = true`なのに `payment_id` が `null`、という話

関数型まつり2026 公募セッション

@nwiizo 50min



nwiizo

株式会社スリーシェイクでプロのソフトウェアエンジニアをやっているものです。SREやクラウドネイティブ技術を専門にしていますが、**型システムと設計の話が好きです。**

趣味は読書、格闘技、グラビア。仕事では型で壁を作り、格闘技では壁を壊そうとしています。

インターネット上では **nwiizo** を名乗り、ブログ「**じゃあ、おうちで学べる**」を運営しています。X / GitHub もこのIDでやっています。

about 3-shake



Mission

インフラをシンプルにしてイノベーションが起こりやすい世界を作る

Vision

労苦（Toil）を無くすサービスを適正な価格で提供し続ける

SRE



日本のSREをリードする
SRE総合支援からセキュリティ
対策を全方位支援

Data Engineering



**あらゆるサービスを
連携するハブになる**
クラウド型ETL/データパイプ
ラインサービスの決定版

Security



**事業者が抱える
セキュリティリスクを無くす**
ワンストップで実現する
セキュリティ対策

HR (Engineer Hiring)



**良いエンジニアに
良い案件を**
フリーランスエンジニアに
「今よりいい条件」を



今日お話しすること

1. なぜ型を「壁」にするのか
2. 関数型の考えをRustで使う
3. 不正な値を作らせない4つのパターン
4. Rustの所有権と型状態を使う
5. DBや既存コードとの境界を設計する
6. まとめ

関数型まつりの参加者には**当たり前**の内容も含まれます。直和型や `Option` / `Result` は短く確認し、**Rustで使うと何が起きるか**に時間を使います。

Rustが初めての方へ：記号は登場時に説明します。コードは、細部より「何を受け取り、何を返すか」を追ってください。



この発表で解決できること

こんな状態に身に覚えはありませんか？

- `is_paid = true` なのに `payment_id` が null
- `status = "verified"` なのに `verified_at` が欠落
- `CustomerId` と `OrderId` が同じ `u64` で取り違い

この発表で持ち帰れるもの

- 型を「壁」として使う設計思想
- 関数型の道具をRustに持ち込む具体パターン
- 所有権・型状態・`PhantomData` でさらに強化する方法
- AI時代でも機械的に検査できる制約の作り方

バグは、対処するものから、書けないものへ



なぜ型を「壁」にするのか

バグを直すのではなく、存在させない

3年運用したDBには、こんなレコードが眠っている

新規のシステムなら、こんなデータは絶対に生まれないと思うかもしれませんが、でも、3年運用したデータベースには、こういう矛盾したレコードが眠っていることがあります。

```
{  
  "order_id": 12345,  
  "is_paid": true,  
  "payment_id": null  
}
```

このレコードは、単体の値だけを見るとそれらしく見えます。 `is_paid` も `payment_id` も、それぞれの型としては正しい。



矛盾はあとから効いてくる

問題は、値を**組み合わせた瞬間**に起きます。

is_paid = trueなのに **payment_id** が null

どの決済で完了したかの記録がなく、返金も、会計との突き合わせもできない。

status = "verified"なのに **verified_at** が null

再認証ポリシーは比較の基準日を持たず、落ちるか素通りするか二択になる。

こうしたレコードを作った瞬間にアラートが鳴ったりはしません。静かにDBに残り、数ヶ月後に返金業務や再認証処理で突然例外を投げる。

単独では正しい値が、組み合わせると矛盾する



なぜ、型はこれを止められなかったのか

先ほどのレコードを受け取る型は、もしかしたらこう書かれていたかもしれません。

```
pub struct Order {  
    pub is_paid: bool,  
    pub payment_id: Option<PaymentId>,  
}
```

この型は、`bool` と `Option<PaymentId>` の**任意の組み合わせ**を許します。「正しい組み合わせ」と「あり得ない組み合わせ」を区別する情報は、型のどこにも書かれていません。

つまり、**型が「正しい形」を規定していない**。正しさはコメントやバリデーション関数の中に散らばり、どこかで抜け落ちる。抜け落ちた瞬間、矛盾レコードが静かに誕生します。



バグを「直す」発想から離れる

不正な状態を見つけたとき、私たちはつい `if` 文で弾き、テストで落とし、レビューで指摘して直します。どれも実行時のチェックや人間の注意力に依存した後追いです。

値そのものは型が許す限り流れ続けます。不正が生まれる根本原因は、手付かずのまま。

起きたバグを追いかけるのは、火が出るたびに消して回る作業。起きえない形を先に作るのは、燃えない素材で建てる設計。どちらも要る。ただし、燃えない素材を知らなければ建てられない。

バグを直すな。表現できなくせよ。

AI Slop問題は、コードにもやってくる

AI Slop は、AIによって作られる低品質なデジタルコンテンツを指す言葉です。ソフトウェア開発でも、生成コードだけでなく、PR・文書・バグ報告にまで及ぶと報告されています。

生成する側

短時間で、もっともらしいコードを大量に作れる。

レビューする側

ドメインの制約や文脈を、1件ずつ検証する必要がある。

生成量が増えるほど、コメントに書いた制約は**有限のレビュー予算**を奪い合います。生成速度にレビューが追いつかなければ、人間は疲弊し、問題が見落とされたまま**低品質なソフトウェアがリリース**されます。

生成コストは下がる。検証コストは消えない

出典: Baltes, Cheong, Treude (2026) “An Endless Stream of AI Slop”



AI時代、コメントより型は破りにくい

では、AI Slopに埋もれない制約はどう作るのか。人間向けの自然言語では、こう書かれます。

```
// payment_id は is_paid=true のときだけ Some、それ以外は None にすること
pub struct Order {
    pub is_paid: bool,
    pub payment_id: Option<PaymentId>,
}
```

コメントは**条件付きのお願い**です。「できればこうしてください」と書いても、人間もAIコーディングエージェントも読み落とすことがあります。

型は**機械が検査する契約**です。「この形でなければビルドを通さない」という制約は、`cargo build` が検査します。

AIはコードを生成できますが、**型エラーのまま動く実行ファイルは生成できません**。だから AI 時代こそ、自然言語の「お願い」を、コンパイラが検査できる「壁」へ移します。

型は、お願いをビルドエラーに変える



壁は、設計図ごと壊せる

型は強い壁ですが、無敵ではありません。壁を迂回する経路は3つあります。

- `unsafe` / `unchecked` API：コンパイラが証明できない安全条件を人間が引き受ける
- 公開フィールドや未検証の `deserialize`：検証付き生成関数を通らない生成経路を残す
- 型定義そのものを緩めるPR：`NonZero` を `u32` に戻す、enumに `Option` を足す

コメントは黙って破れますが、型の壁を迂回すると痕跡が残ります。`unsafe` や `unchecked` API は呼び出しとして、型定義の変更は `diff` として可視化される。

型は壁ですが、壁の設計図を守るのは人間のレビューです。AI が型定義を緩める PR を出したとき、気づけるのは人間だけ。型はコードレビューを不要にはしません。

型は壁。でも、壁の設計図を守るのは人間



なぜ壁を築くかはわかった

では、何で築くのか？

関数型の道具をRustに持ち込む

型は、値の集まりに付ける名前

関数型まつりの皆さんにはおなじみですが、ここでは Rust の例で確認します。

型 = 値の集合に付けた名前 (`bool` = {true, false}、`u64` = 0～約1800京)

型を「値の集まり」と考えると、設計の狙いが単純になります。**正しい値だけが入るように、集合を小さくする**のです。

たとえば「0ではない整数」という型を作れば、0はその集合に入りません。0を見つけて弾くのではなく、**最初から表現できなく**します。

型を狭くすると、バグの置き場所も狭くなる



AND型は掛け算、OR型は足し算

データの形は、全部を持つか、どれか1つを選ぶかで考えます。

AND型 (struct)

フィールドが全部同時に存在する。

```
struct User {  
    name: String,  
    age: u32,  
}
```

OR型 (enum)

いくつかの候補からちょうど1つ。

```
enum Shape {  
    Circle(f64),  
    Square(f64),  
}
```

`Option` (あるか、ないか) と `Result` (成功か、失敗か) もOR型です。AND型は組み合わせが増えるので掛け算、OR型は候補を並べるので足し算になります。

掛け算で増えた組み合わせに、バグが潜む



関数型の道具は、Rustにある

ここでは、関数型でおなじみの道具がRustでは何に当たるかだけ整理します。

関数型 (F# / OCaml / Scala など)	Rust
Record / Sum type (Choice, DU)	<code>struct</code> / <code>enum</code>
<code>Maybe</code> / <code>Either</code>	<code>Option<T></code> / <code>Result<T, E></code>
Pattern matching (網羅性チェック)	<code>match</code> (網羅をコンパイラが強制)
Single case DU + Smart constructor	タプル構造体 + <code>pub fn new() → Result</code>

`struct`、`enum`、網羅的なパターンマッチが揃っているので、関数型のドメインモデリングは**ほぼそのままRustで書けます**。ここからは、Rust固有の所有権も組み合わせます。

Rust豆知識 (Programming Rust 3rd Early Release, Ch.10)

Rustの `enum` は、ML系言語の直和型に相当します。Rustでは、そこへ参照・可変性・メモリ安全性を検査する借用チェッカーも組み合わせられます。



Rustの記号は、ここだけ読めればいい

本編で頻繁に登場する記号だけ先に整理します。これでコード例は雰囲気を読めます。

参照と所有権

- `&T` : 読み取り専用で借りる
- `&mut T` : 書き換え可能な状態で借りる
- 参照なし (`T`) : **所有権ごと渡す**。呼び出し元では使えなくなる

エラー処理

- `Result<T, E>` : 成功 `Ok(T)` か失敗 `Err(E)`
- `?` 演算子 : 失敗ならその場で返し、成功なら中身を取り出す
- `Option<T>` : 値がある `Some` か、ない `None`

細かい記号は登場時に補足します。構文に悩んだら「型が何を受けて何を返すか」だけ見てください。



? は、失敗した時点で処理を返す

? は、処理に失敗したらその場で呼び出し元へ返し、成功したら中身を取り出します。

```
fn parse_user_id(s: &str) → Result<UserId, ParseError> {  
    // (1) 文字列を数字にパース。失敗すれば Err で早期リターン  
    let n: u64 = s.parse()?;  
  
    // (2) UserId を作る。0 は不正なので失敗しうる  
    let id = UserId::new(n)?;  
  
    // (3) 成功したら Ok で包んで返す  
    Ok(id)  
}
```

Option<T> と Result<T, E> は、どちらも ? でつながられます。失敗のたびに if を書く必要はありません。



所有権は、古い状態の使い回しを止める

Rustの**所有権**は、「各値には所有者がいる」というルールです。家の鍵を誰かに渡すと、自分の手元には残りません。それと同じように、`Copy` でない値を関数へ渡すと所有権が移り、呼び出し元からは使えなくなります。

この仕組みを使うと、「検証前の注文は、検証した後に残ってはいけない」を型で表現できます。「検証前の注文をもう1回使う」コードが、**そもそも書けません**。前の状態が残らないことを、所有権ルールが型レベルで保証してくれます。

このルールが、後半で「状態遷移の壁」になります。コードはパターン1でお見せします。



Rustの値は、標準では書き換えられない

Rust では変数も参照も、デフォルトで書き換え不可です。書き換えたいときは `mut` を明示する必要があります。

```
let order = ValidatedOrder { items: vec![ ... ] };
order.items.push(new_item);    // ← コンパイルエラー。order は不変

let mut order = ValidatedOrder { items: vec![ ... ] };
order.items.push(new_item);    // ← これは通る
```

この仕組みは、メソッドの設計にも影響します。関数型と同じく「入力を受けて新しい値を作る」が自然になります。

```
// 関数型的な書き方: 新しい値を返す
fn apply_discount(o: PricedOrder, rate: f64) → PricedOrder { /* ... */ }

// OOP 的な書き方: 既存の値を書き換える
impl PricedOrder {
    fn apply_discount(&mut self, rate: f64) { /* ... */ }
}
```

前者は `&mut self` を要求せず、入力の所有権を受け取って次の値を返します。その場で書き換えるAPIではないことが関数の型に現れます。



線形型とアフィン型は、使える回数が違う

値を何回使えるかに注目すると、Rustの所有権を整理できます。

線形型 (Linear types)

値は**ちょうど1回**使います。捨てることも複製すること
もできません。

代表例: Clean の uniqueness types、Idris 2 の linear types

アフィン型 (Affine types)

値は**1回以下**使います。複製はできませんが、**捨てるこ
とはできます**。

代表例: Rust の所有権、ATS、Vale

Rustの通常の所有値は、捨ててもよいアフィン型の側です。



Rustの所有値は、高々1回だけ使える

`Copy` ではない値を関数へ渡すと、所有権が移ります。同じ値をもう一度使おうとすると、コンパイルエラーになります。

```
let order = UnvalidatedOrder::new();
validate(order)?; // ここで所有権が移る
validate(order)?; // エラー。order はもう使えない
```

一方、使わずに捨てることはできます。`Copy` 型は複製できる例外で、`#[must_use]` は捨てた値を警告する仕組みです。

この「使い回せない」という制約が、古い状態を残さない壁になります。



道具は揃った

では、どう使うのか？

状態を分ける、意味を分ける、作り方を絞る、正しい組み合わせだけ残す

パターン1 状態が違えば、型を分ける

「検証前の注文」と「検証済みの注文」を**同じ型**で扱うと、検証をスキップしたコードが通ります。

```
struct Order {  
    validated: bool,  
    items: Vec<Item>,  
}  
  
fn calculate_total(order: &Order) → Money {  
    // validated = false でも呼べてしまう  
    order.items.iter().map(|i| i.price).sum()  
}
```

`validated` フラグは実行時の情報であり、呼び出し側が確認を忘れてもコンパイラは何も言いません。



別の型にすれば、順序違反は書けない

検証前と検証後を別の型にします。

```
struct UnvalidatedOrder { items: Vec<Item> }
struct ValidatedOrder   { items: Vec<Item> }

fn validate(o: UnvalidatedOrder) → Result<ValidatedOrder, OrderError> {
    // チェックを通った場合だけ ValidatedOrder が生まれる
}

fn calculate_total(o: &ValidatedOrder) → Money {
    o.items.iter().map(|i| i.price).sum()
}
```

`calculate_total(&unvalidated)` はコンパイルエラーになります。「検証前の注文を価格計算に渡す」バグは、そもそも書けません。



ワークフロー全体を型で貫く

状態ごとに型を分ける発想を注文処理の一連の流れに適用すると、こうなります。

```
struct UnvalidatedOrder { items: Vec<RawItem> }
struct ValidatedOrder   { items: Vec<Item>   }
struct PricedOrder      { items: Vec<Item>, subtotal: Money }
struct PaidOrder        { items: Vec<Item>, subtotal: Money, payment: PaymentId }

fn validate(o: UnvalidatedOrder) → Result<ValidatedOrder, OrderError>;
fn price(o: ValidatedOrder)      → PricedOrder;
fn charge(o: PricedOrder, card: &Card) → Result<PaidOrder, PaymentError>;
```

各ステップの入出力が型で固定されているので、**順序を間違えるコードは書けません**。検証前の注文に価格をつけるコードも、支払い前の注文を確定するコードも、コンパイルが通りません。

関数の入出力の型が、ワークフローの仕様書になる



ワークフローは左から右へ流せる

状態型で固めた各ステップは、関数合成でそのまま繋がります。tap クレートの `.pipe()` を挟むと、ネストせず左から右に読める形になります。

```
let paid = raw
  .pipe(validate)           // Unvalidated → Result<Validated>
  .map(price)               // Validated    → Priced
  .map_err(WorkflowError::from)
  .and_then(|o| charge(o, &card).map_err(WorkflowError::from));
```

パイプの途中で型が合わなければ、その行が赤くなります。合成できる順序はコンパイラが保証する。関数型の「データを変換で流す」発想が、状態遷移にそのまま乗ります。



パターン2 同じ数字でも、意味ごとに型を分ける

状態型（パターン1）とその応用はここまで。残り3つのパターンに進みます。まずは**取り違いの防止**。

顧客IDと注文ID、どちらも `u64` で扱うと、引数の順序ミスがすり抜けます。

```
fn charge(customer_id: u64, order_id: u64) { /* ... */ }  
  
let customer = 1001u64;  
let order = 5678u64;  
  
charge(order, customer); // ← 引数逆でもコンパイルが通る
```

これは**実行時に初めて気づくバグ**です。ユニットテストを全網羅しない限り、本番で発覚します。



newtypeは、値を1つだけ包む

Rust本でも紹介されているニュータイプパターンです。タプル構造体で1フィールドだけ包みます。

```
#[repr(transparent)] struct CustomerId(u64);
#[repr(transparent)] struct OrderId(u64);

fn charge(customer: CustomerId, order: OrderId) { /* ... */ }

let customer = CustomerId(1001);
let order = OrderId(5678);

charge(order, customer); // ← 型エラー、コンパイルが通らない
```

`#[repr(transparent)]` を付ければ `u64` と同じレイアウトが保証され、取り違えはコンパイラが検出してくれます。

Rust豆知識 (Programming Rust 3rd Early Release, Ch.9)

書籍では、コメントで意味を区別する代わりにニュータイプを使い、Rustの型検査へ任せる方法が紹介されています。意味が違値を、同じ基本型へ戻さないのがポイントです。

同じ `u64` に別の意味を持たせると、必ずどこかで混ざる



パターン3 作り方を限定して、不正値を作らせない

この方法は**検証付き生成関数 (Smart Constructor)** と呼ばれます。「メールアドレスには @ が必要」という条件を、値を作るときに必ず検査します。

```
pub struct Email(String);

impl Email {
    pub fn new(s: &str) → Result<Self, EmailError> {
        if !s.contains('@') { return Err(EmailError::Invalid); }
        Ok(Email(s.to_owned()))
    }

    pub fn as_str(&self) → &str { &self.0 }
}
```

中身のフィールドは**非公開**なので、外部コードは `Email::new` を通らずに `Email` 値を作れません。この例では、一度作れた `Email` は以降のコードで必ず `@` を含みます。



複数の制約を組み合わせる

実務では複数の制約を束ねることがよくあります。

```
pub struct CustomerName { first: String, last: String }

impl CustomerName {
    pub fn new(first: &str, last: &str) → Result<Self, NameError> {
        let first = first.trim();
        let last = last.trim();
        if first.is_empty() || last.is_empty() {
            return Err(NameError::Empty);
        }
        if first.chars().count() > 50 || last.chars().count() > 50 {
            return Err(NameError::TooLong);
        }
        Ok(CustomerName { first: first.into(), last: last.into() })
    }
}
```

このコンストラクタを通った `CustomerName` は、空でも長すぎもしないことが型で保証されます。



検証した印を、型に残す

検証付き生成関数の考え方を、Alexis Kingは「Parse, don't validate」と表現しました。

"Parse, don't validate."

検証するな。解釈せよ。

validate は `bool` を返すだけ。通った値も通らなかった値も、同じ `String` のまま旅を続けます。**parse** は別の型に変換し、「検証済みである」情報が型に刻まれます。下流のコードは、もう検証を気にしません。

```
fn validate(s: &str) → bool;           // 情報は型に残らない
fn parse(s: &str) → Result<Email, _>;  // 情報が型に刻まれる
```

検証済みの証明を、値そのものに運ばせる



この型、4通りのうち何通りがバグ？

1つの値は、検証付き生成関数で守れました。次は**値の組み合わせ**です。「認証済みなら認証日時がある」をstructで書くと、何通りの状態が生まれるでしょうか。

```
struct User {  
    email: String,  
    is_verified: bool,  
    verified_at: Option<DateTime<Utc>>,  
}
```

この型が表現できる状態は、**4通り**です。

is_verified	verified_at	意味
false	None	未認証（正しい）
true	Some(t)	認証済み（正しい）
true	None	不正：認証済みなのに日時がない
false	Some(t)	不正：未認証なのに日時がある



型の半分が、バグの置き場所になる

4通りのうち、業務上正しいのは2通りだけです。

型が許す状態

`bool` 2通り × `Option` 2通り
= 4通り

業務で正しい状態

未認証 + 認証済み
= 2通り

AND型は、フィールドの選択肢を掛け算します。フラグを足すたびに状態は増えてますが、**増えた状態が正しいとは限りません。**

正しい状態が2つなら、型にも2つだけ書く



正しい状態だけをenumに書く

フラグと日時を別々に持たず、「未認証」か「認証済み」のどちらか一方を `enum` で表現します。

```
enum User {  
    Unverified { email: String },  
    Verified   { email: String, verified_at: DateTime<Utc> },  
}
```

`Verified` なら `verified_at` は必ず存在します。 `Unverified` には、そのフィールド自体がありません。

4通りを許してから2通りを弾くのではなく、正しい2通りだけを最初から型に書きます。



matchは、状態の書き忘れも防ぐ

enumを使うコードは、`match` で状態ごとの処理を書きます。

```
fn send_receipt(user: &User) {  
    match user {  
        User::Verified { email, verified_at } => { /* 送る */ }  
        User::Unverified { .. } => { /* 送らない */ }  
    }  
}
```

どちらかの状態を書き忘れると、コンパイラがエラーにします。新しい状態を追加したときも、修正が必要な
`match` がすべて分かります。

不正な状態を表現不可能にする



型は、コンパイル時のユニットテストになる

enumで「ありえない状態」を書けなくすると、**もう1つ利点**があります。

「認証済みなのに `verified_at` が `null` のケース」のテストは、書く必要がなくなります。

なぜなら、そのケースは**コンパイラが通さない**からです。

この仕組みを、Scott Wlaschin は「**コンパイル時のユニットテスト**」と呼びます。型が、実行前にテストの役目を果たしてくれるのです。テストコードが不要になるのではなく、**型そのものがテスト**になります。



4つの基本パターン、一覧で確認

ここまで駆け足で見てきた4パターンを、一枚にまとめます。

1. 状態ごとに型を分ける

`UnvalidatedOrder` と `ValidatedOrder` は別の型。検証前の注文を価格計算に渡すコードは、書けない。

2. 意味ごとにnewtypeを作る

`CustomerId` と `OrderId` を別の型に。引数の順序を間違えればコンパイルが通らない。

3. 検証付き生成関数を通す

作り方を限定すれば、不正な `Email` は存在できない。検証済みという情報が型に残る。

4. 正しい組み合わせだけをenumにする

フラグと `Option` を別々に持たず、正しい状態だけを候補として並べる。

この4つだけでも、『`is_paid` と `payment_id` が矛盾する』系のバグは書きようがなくなります。



4つの基本パターンは揃った

ここから、Rustの型機能でさらに強くする

型状態 / PhantomData / 既製の制約型

状態を型パラメータに乗せる

先ほどは `UnvalidatedOrder` と `ValidatedOrder` を別々の構造体にしました。中身がほぼ同じなら、注文は1つのまま、**状態だけを型で切り替えられます**。

型状態 (Type State) パターン

`Order<Unvalidated>` と `Order<Validated>` は、中身が同じでも**別の型**です。検証が終わるまで、検証済みの注文だけに許した操作は呼びません。

別々の構造体を何度も書かずに、状態遷移を型で表せるのが利点です。



PhantomDataは、実体を増やさず状態を載せる

`Order<State>` と書いても、注文のフィールドには `State` の値がありません。そこで `PhantomData` を使い、この型は`State`に依存するとコンパイラへ伝えます。

```
use std::marker::PhantomData;

struct Order<State> {
    items: Vec<Item>,
    _state: PhantomData<State>,
}
```

`PhantomData<State>` は実行時には0バイトです。データを増やさず、型の上だけに状態を追加します。



型状態パターンのコード例

```
use std::marker::PhantomData;

struct Unvalidated; struct Validated;
struct Order<State> { items: Vec<Item>, _state: PhantomData<State> }

impl Order<Unvalidated> {
    fn validate(self) → Result<Order<Validated>, OrderError> {
        Ok(Order { items: self.items, _state: PhantomData }) // 検証は省略
    }
}

impl Order<Validated> {
    fn total(&self) → Money { // Validated にだけ実装
        self.items.iter().map(|i| i.price).sum()
    }
}
```

`total()` は `Order<Validated>` にしか実装されていないので、`Order<Unvalidated>` に対しては呼びません。状態遷移は `self` 消費で表現され、`Unvalidated` の注文は検証後には残りません。



自作する前に、既製の制約型を探す

ここまでの壁は全部自作でした。でも、自作する前に**すでに用意されている制約型**で済まないかを確認するのが先です。標準ライブラリとエコシステムに、制約が型に埋め込まれた型はいくつも揃っています。

標準ライブラリ

- `NonZero<T>` : ゼロではない整数型
(`NonZero<u32>`、`NonZeroU32` は別名)
- `Option<&T>` : 参照自体は非null。値がない場合は `None`
- `String` / `&str` : UTF-8を保証

エコシステム

- `nonempty` : `NonEmpty<T>` で空でないVecを保証
- `nutype` : マクロでnewtypeと検証処理をまとめて生成
- `bounded-integer` : 範囲を型に持つ整数newtypeを生成

`n: u32` ではなく `n: NonZero<u32>`、`items: Vec<Item>` ではなく `items: NonEmpty<Item>` と書けば、「ゼロ・空を渡してはいけない」が型に残り、生成後のAPIでは再確認が不要になります。



公開APIの拡張点を、型で閉じる

型は壁ですが、公開APIの拡張点には穴が残ります。 `enum` はもともと閉じていますが、 `trait` は放っておくと外部のコードから実装を増やせます。

そこで `trait` を非公開の `supertrait` で封じます (`sealed trait`)。

```
mod sealed { pub trait Sealed {} }

pub trait PaymentState: sealed::Sealed { /* ... */ }

pub struct Authorized;
impl sealed::Sealed for Authorized {} // 実装できるのは自分のクレートだけ
impl PaymentState for Authorized { /* ... */ }
```

`sealed::Sealed` が非公開なので、外部クレートは `PaymentState` を実装できません。「どこまで拡張を許すか」を、お願いではなく型で決められます。



フィールドを足すとOptionの海に戻る

要件が増えたとき、つい既存の型にフィールドを足したくなります。でもそれは、組み合わせ爆発とOptionの海に戻る道です。

```
// Before: ValidatedOrder に配送料の情報を足したい
struct ValidatedOrder {
  items: Vec<Item>,
  shipping_cost: Option<Money>, // ← 計算前は None
  shipping_address: Option<Address>, // ← 計算前は None
}
```

配送料が計算済みかどうか、配送先が確定しているかどうか、また `Option` の組み合わせに散らばります。



設計を進化させるときも、型を作る

新しい状態は、新しい型にする。配送情報が必要になったら、それを必須フィールドに持つ型を作ります。

```
struct PricedOrder { items: Vec<Item>, subtotal: Money }
struct PricedOrderWithShipping { // 配送情報は必須・Optionにしない
    items: Vec<Item>, subtotal: Money, shipping: ShippingInfo,
}
```

型が増えると、コンパイラが依存箇所を**全部追跡**します。「直し忘れ」はビルドエラーになる。`Option` で様子を見るより安全です。

ここまでは型で何でも解決できそうに見えます。でも、**Rust固有の摩擦**もあるのが現実です。



非純粋な現実と向き合う

所有権・境界・DB・既存コード

摩擦1 古い状態を借りたまま、次へ進めない

ここまで型で何でも守れるかのように話してきました。でも、実際にコードを書くと、最初につまずくのがここです。

状態遷移を `fn validate(o: UnvalidatedOrder) → Result<ValidatedOrder, _>` の形で書くと、入力の注文は**所有権ごと消費**されます。

これが摩擦になるのは、古い注文への参照を残したまま、次の状態へ進めたい場面です。

```
let order: UnvalidatedOrder = receive();

let audit_view = &order;           // 古い状態を借りる
let valid = validate(order)?;      // ← 借用中なので move できない
log_for_audit(audit_view);        // 借用がここまで生きている
```

監査ログやメトリクスへ古い値を渡したいなら、必要な情報だけコピーするか、共有したい部分を `Arc` に分けます。古い状態を誰がいつまで見るのかを明示的に決める必要があります。



共有したい中身だけ、構造共有する

注文全体ではなく、大きなコレクションだけを新旧の状態でも共有したいなら、**永続データ構造**が使えます。 `im-rc` の `Vector` / `HashMap` は、更新しても古い値を残し、新しい値を返します。

```
use im_rc::Vector;

fn add_item(items: &Vector<Item>, new: Item) → Vector<Item> {
    let mut next = items.clone(); // clone は構造共有なので安価
    next.push_back(new);          // next だけが伸びる
    next                          // 元の items は不変のまま残る
}
```

この `clone()` は全要素を複製せず、変更していない部分を新旧の値で共有します。これを**構造共有**と呼びます。型状態の保証を保ったまま、複製コストを抑えられます。



摩擦2 重複を消すと、matchが1段深くなる

パターン4の enum では、共通の `email` が両方のバリエーションで繰り返されていました。Rust には「enum 全体で共通のフィールド」を直接書く機能はありません。対処は2つあります。

素直に繰り返す

```
enum User {
    Unverified {
        email: String,
    },
    Verified {
        email: String,
        verified_at: DateTime<Utc>,
    },
}
```

状態だけを enum にする

```
struct User {
    email: String,
    state: UserState,
}
enum UserState {
    Unverified,
    Verified(DateTime<Utc>),
}
```

どちらも「検証済みのときだけ `verified_at` がある」保証は同じです。後者は共通フィールドを1箇所に集められる代わりに、状態を見るたび `user.state` を1段深く `match` します。トレードオフは保証の強さではなく、重複と操作の近さです。



摩擦3 型状態は、読む人の負担を増やす

`Order<Unvalidated>` と `Order<Validated>` で状態を分けると壁は強固になりますが、関数の型が複雑になります。

```
// 2状態なら、まだ読める
fn process(o: Order<Validated>) → Result<Order<Priced>, Error>;

// 3状態以上で、複数のジェネリクスが絡むと急に重くなる
fn handle<S>(o: Order<S>) → Result<Output, Error>
where
    S: Into<FinalState>,
    Order<S>: Processable;
```

型が表す制約は強くなりますが、読み手には「この `S` は何か」「どの状態へ進むのか」を追う負荷が増えます。



型状態を使う境界を決める

関数型まつりの皆さんには馴染みがあっても、**チームの全員がこの関数の型を読めるとは限りません**。制約を増やすほど、読む負担も増えます。

- 順序違反をコンパイル時に止めたいワークフローなら型状態パターン
- 実行時に任意の状態を読み込むなら enum、状態ごとに中身が大きく違うなら別 struct
- 共有データが大きいなら、状態と共通データを分離して読む負担と複製コストを抑える

チームが読めない壁は、守りの価値が逆転する



外から来る値には、まだドメイン型がない

型で守れるのは、業務ルールを扱うドメインの内側です。HTTPやDBから来た直後の値には、まだ専用の型が付いていません。

```
#[derive(Deserialize)]                // HTTP入力を受ける境界型
struct CreateOrderRequest {
    customer_id: u64,                  // ← まだ CustomerId ではない
    items: Vec<ItemInput>,             // ← まだ ValidatedItem ではない
    email: String,                     // ← まだ Email ではない
}
```

HTTP、JSON、DB、メッセージキューから来る値は、最初は `u64` や `String` です。内側へ渡す前に、`CustomerId` や `Email` へ変換します。



境界で型に変換し、内側へ渡す

```
fn create_order(req: CreateOrderRequest) → Result<ValidatedOrder, ApiError> {  
    let customer = CustomerId::new(req.customer_id)?;  
    let email     = Email::new(&req.email)?;  
    let items     = req.items.into_iter().map(Item::try_from)  
                        .collect::<Result<Vec<_>, _>>()?;  
    ValidatedOrder::new(customer, email, items)  
}
```

境界のただ1か所で検証して型に変換します。以降は、型付きの値だけが内側を流れます。

境界で型を貼り、内側は型で守る



DBへ保存するとき、状態は列に戻る

リレーショナルDBの行は**列の組**です。Rustのenumを保存するときは、`is_paid BOOLEAN` と、NULLを許す `payment_id UUID` のような列へ戻すことがあります。

Rust 側では決済状態を `enum` で受けられますが、**書き込み時には再びフラグとnullに分解**する必要があります。

```
// enumからDBの列へ変換する
fn to_row(payment: &PaymentState) → OrderRow {
    match payment {
        PaymentState::Unpaid ⇒ OrderRow {
            is_paid: false,
            payment_id: None,
        },
        PaymentState::Paid(id) ⇒ OrderRow {
            is_paid: true,
            payment_id: Some(id.clone()),
        },
    }
}
```



DBの矛盾は、読み込み時にエラーにする

DBから読み込むときは、列の組をもう一度ドメイン型へ変換します。

```
fn from_row(row: OrderRow) → Result<PaymentState, OrderRowError> {  
    match (row.is_paid, row.payment_id) {  
        (false, None)    ⇒ Ok(PaymentState::Unpaid),  
        (true, Some(id)) ⇒ Ok(PaymentState::Paid(id)),  
        (true, None)    ⇒ Err(OrderRowError::MissingPaymentId),  
        (false, Some(_)) ⇒ Err(OrderRowError::UnexpectedPaymentId),  
    }  
}
```

冒頭の「3年眠っていたレコード」と再会するのは、まさにこの `(true, None)` の行です。型は新しい矛盾を防ぎ、すでにある矛盾はここで名前付きのエラーとして表面化する。

読み込み時は不正な組み合わせを `Err` にし、DB側にも `CHECK` 制約を置いて新しい矛盾を拒みます。既存DBの形をそのままドメイン型へ持ち込まず、境界で相互変換するのがポイントです。



既存コードへの段階導入

u64 が配り回っているコードベースに newtype を一気に入れるのは大工事です。影響範囲が全ファイルに及ぶこともあります。

現実解は、**境界から小さく導入**することです。

1. **API入口**：HTTPハンドラや deserialize 直後で newtype に変換
2. **リポジトリ層**：DB から取り出した直後、返す型を newtype に
3. **内側に染み込ませる**：呼び出されている関数の引数・戻り値の型を順次 newtype に置き換え

すべてを一度に直す必要はありません。**境界に置いた newtype が、内側の型付けを徐々に強制していく**のが実務的なパターンです。



型エラーは、書いた瞬間に見つける

所有権、境界、DB、既存コードには、それぞれ導入の摩擦がありました。続けやすくするには、**型エラーへすぐ気づける**ことが重要です。

不正な状態を型で表現できなくしても、エラーに気づくのが遅ければ使いづらくなります。**rust-analyzer**はコンパイラの診断をエディタへ届け、編集集中に赤線を表示します。

rust-analyzer が壁に効く理由

- 編集集中に型エラーを表示
- newtype / enum の `match` 漏れも診断として表示
- IDE の上で「書けないコード」が分かる

型の壁は、CI で初めて効くより、エディタで書いた瞬間に効くほうが強い。フィードバックが早いほど、壁は設計の一部になります。



型で書けない規約は、自動検査する

rowanは、ソースコードをコメントや空白も含めて木構造で扱うライブラリです。コードの形を調べられるので、チーム固有の規約を検査するlintの土台にできます。

rowanで検査できる例

- `pub ...id: String` → newtype に矯正
- `is_paid: bool` + `payment_id: Option` → enum 化を促す
- `pub enum ...Error` に `#[non_exhaustive]` 必須

`cargo check` が知らない**チーム固有の規約**も、lintなら自動で検査できます。機械に任せる範囲を増やすほど、人間は仕様そのものが正しいかの確認に集中できます。

CIだけで弾くな。書いた瞬間にも弾け



型で防げること、防げないこと

ここまでの話を踏まえて、**型の守備範囲**を整理します。

型で防げること

- 「検証前の値」を「検証済みの値」として扱うこと
- 同じ表現の別の概念の取り違い
- ありえない組み合わせの同時存在
- コメントだけに頼った不変条件の流出

型で防げないこと

- ビジネスロジックの間違い（計算式の誤り等）
- 外部システムの値の変化
- パフォーマンスや可用性の問題
- 仕様の理解違い

型が防ぐのは「**不正な状態が作れてしまうこと**」であり、「**何が正解か**」を決めるのは人間です。この区別を忘れると、型原理主義に陥って苦しくなります。



通っても、正しいとは限らない

AI に「定員に空きがあれば参加登録する」と頼むと、こんな素直なコードが出ます。型も単体テストも通ります。

```
let count = participant_count(event_id)?;           // 取得
if count < capacity {                               // 確認
    insert_participant(event_id, user_id)?;         // 登録
}
```

でも残席1のとき二人がほぼ同時に申し込むと、どちらも確認を通過して2件とも登録されます。確認と登録の間に割り込めるからです。並行性やトランザクション整合性は**この型だけでは表現できない**、システム全体の制約です。

文脈を**名前や型に込めれば**、AI もそこから推論できるようになります。それでも、何を正しい振る舞いとするかを規定し、最後に検証する責任は、**外側を知る人間**に残ります。

型は形を守る。正しさは、文脈に宿る



この考え方は、Rustに限らない

ここまで、型が守れる範囲と限界まで見てきました。最後に、この設計をRustの外へ広げます。「不正な値を型で表現できなくする」という発想は、多くの言語で使えます。

関数型言語

- F# / OCaml: Discriminated Union
- Scala 3: `enum` (ADT 構文)
- Elm: Custom Types
- ReScript: Variants

マルチパラダイム言語

- TypeScript: discriminated union + literal types
- Kotlin: `sealed class` / `sealed interface`
- Swift: `enum` with associated values
- Java: `sealed interface` (Java 17+)

持ち帰っていただきたいのはRustの構文ではなく、**正しい値だけを型で表す設計**です。



まとめ

型は壁であり、お願いではない

今日の要点

設計思想

- 型はコンパイラのための注釈ではなく、不正な状態を物理的に禁止する壁
- バグは直すのではなく存在させない
- AIが書く時代も破れない壁は型。正しさが宿る文脈の検証は人間に残る

Rustで壁を築く道具立て

- 基本4パターン：状態を分ける、意味を分ける、作り方を絞る、正しい組み合わせだけ残す
- Rust固有：型状態 / `PhantomData` / 既製の制約型
- 進化のとき：フィールドを足さず、新しい型を作る
- 境界：外から来た値を型に変換し、内側は型で守る

バグを直すな。表現できなくせよ。



参考資料① 書籍と記事

- **Programming Rust, 3rd Edition (Early Release)** (Jim Blandy, Jason Orendorff, Leonora F. S. Tindall 著, O'Reilly, Rust 2024 Edition対応, 2026年7月参照)
 - 第9章 構造体 (タプル構造体とニュータイプ)
 - 第10章 列挙型とパターン (代数的データ型と `match`)
- **Domain Modeling Made Functional** (Scott Wlaschin 著, Pragmatic Bookshelf, 2018)
 - 第4章 Understanding Types (choice types / discriminated unions)
 - 第5章 Domain Modeling with Types (Constrained Values, Modeling with Choice Types)
 - 第6章 Integrity and Consistency in the Domain (smart constructors / 不正な状態を表現不可能にする / "compile-time unit tests")
- **"Designing with Types" シリーズ** (Scott Wlaschin, F# for Fun and Profit, 全13回)
 - 代表記事: 「不正な状態を表現不可能にする」
 - シリーズ目次: fsharpforfunandprofit.com/series/designing-with-types/
- **"Parse, don't validate"** (Alexis King, 2019)
 - lexi-lambda.github.io/blog/2019/11/05/parse-don-t-validate/
- **"Rustでも学べる関数型ドメイン駆動設計"** (nwiizo, 2026)
 - syu-m-5151.hatenablog.com



参考資料② 理論的背景とツール

- 線形型・アフィン型の理論的背景

- Girard (1987) "Linear Logic". 線形論理の原典（線形型の理論的源流）
- Bernardy, Boespflug, Newton, Peyton Jones, Spiwack (2018) "Retrofitting Linear Types" (POPL'18)。既存の言語に線形型を後付けする設計論文
- Walker (2005) "Substructural Type Systems"（部分構造型システム概観）

- 型を IDE と Linter で常時強制する基盤

- **rust-analyzer**: github.com/rust-lang/rust-analyzer。IDE向けRust解析サーバ。型エラーや `match` 漏れをリアルタイムで弾く
- **rowan**: github.com/rust-analyzer/rowan。rust-analyzerでも使われるロスレス構文木ライブラリ。構文解析層と組み合わせて自家製lintの土台にできる

補足: 「不正な状態を表現不可能にする」という表現は、Yaron Minsky の "Effective ML" で提示され、Scott Wlaschin が F# とドメインモデリングの文脈で広く紹介しています。



どの型から書き始めますか？

変えるのは、型の形ではなく、発想の形

`is_paid = true`なのに `payment_id` が `null`。あのレコードを、もう作れなくする型から。

ありがとうございました

バグを直すな、書けないコードにせよ

どうしても書けたら、型が足りない

@nwiizo | <https://3-shake.com>